

DroneWar

Dokumentace semestrální práce

Autor: Tomáš Svoboda

Předmět: KIV/UPG

Datum: 3. prosince 2025

Obsah

1	Úvod	3
2	Struktura projektu	3
2.1	Adresář Models/	3
2.2	Kořenový adresář	3
3	Klíčové algoritmy pohybu	4
3.1	Výpočet trajektorie střely	4
3.2	Výpočet počáteční rychlosti střely	5
3.3	Pohyb dronu	6
3.4	Bilineární interpolace výšky	7
3.5	Algoritmus predikce dopadu	8
4	Vykreslení	9
4.1	Dělo	9
4.2	Dron	9
4.3	Mapa terénu	10
4.4	Spodní profil a grafy	11
4.4.1	Korekce vizuálního úhlu hlavně	11
4.4.2	Terén	11
4.5	Vítr	12
4.6	Projektil	12
5	Uživatelské rozhraní a ovládání	12
6	Instalace a spuštění	13
7	Implementovaná rozšíření	14
7.1	Proměnlivý vítr	14
7.2	Realistické modely (SVG)	14
7.3	Kráter	14
7.4	Animace exploze	15
8	Použité zdroje a nástroje	16
8.1	Knihovny a Frameworky	16
8.2	SVG modely	16

1. Úvod

DroneWar je aplikace vytvořená jako semestrální práce z předmětu KIV/UPG. Hlavním cílem projektu je implementovat vizualizaci balistické střelby s realistickou fyzikou a interaktivním prostředím, včetně:

- výpočtu trajektorie střely pod vlivem gravitace a větru,
- animace pohybu nepřátelského dronu směrem k dělu,
- detekce kolizí střely s dronem a terénem,
- interaktivní vizualizace výškového profilu a ovládání děla.

2. Struktura projektu

2.1. Adresář Models/

Adresář Models/ obsahuje logiku simulace a jednotlivé třídy reprezentující fyzikální objekty:

Drone.cs	Třída reprezentující nepřátelský dron
Cannon.cs	Třída reprezentující dělo
Projectile.cs	Třída reprezentující střelu vystřelenou z děla
Wind.cs	Třída reprezentující vítr s proměnlivou silou
ScenarioData.cs	Data načtená ze scénáře a výpočty nad terénem
ScenarioLoader.cs	Třída pro načítání scénářů
TerrainMap.cs	Generování barevné výškové mapy
TerrainProfile.cs	Vykreslování řezu terénem a grafů trajektorie

2.2. Kořenový adresář

Kořenový adresář obsahuje zbylé komponenty:

MainWindow.axaml/.cs	Hlavní okno, zpracování vstupů (klávesnice/myš)
BattleField.axaml/.cs	Herní okno s vizualizací bojiště a animací
Program.cs	Vstupní bod aplikace, načtení čísla scénáře
App.axaml.cs	Konfigurace aplikace, definice stylů

3. Klíčové algoritmy pohybu

3.1. Výpočet trajektorie střely

Nejdůležitějším algoritmem aplikace je výpočet balistické trajektorie střely. Program počítá pozici střely po každém časovém kroku a vykresluje ji.

Trajektorie se počítá numerickou integrací s pevným časovým krokem $\Delta t = 0,01$ s podle následujících diferenčních rovnic:

$$\mathbf{p}_{t+\Delta t} = \mathbf{p}_t + \mathbf{v}_t \cdot \Delta t \quad (1)$$

$$\mathbf{v}_{t+\Delta t} = \mathbf{v}_t + (0, 0, -g) \cdot \Delta t + (\mathbf{v}_w - \mathbf{v}_t) \cdot b \cdot \Delta t \quad (2)$$

kde:

- $\mathbf{p}_t$ je trojrozměrný polohový vektor střely v čase t
- $\mathbf{v}_t$ je trojrozměrný vektor rychlosti střely v čase t
- $\mathbf{v}_w$ je trojrozměrný vektor rychlosti větru
- $g = 10 \text{ m} \cdot \text{s}^{-2}$ je gravitační zrychlení
- $b = 0,05 \text{ s}^{-1}$ je koeficient odporu vzduchu / vlivu větru

První rovnice je implementována ve funkci `UpdatePosition`, která aktualizuje souřadnice objektu podle aktuální rychlosti:

```
1 FUNKCE UpdatePosition(deltaTime):  
2     x = x + velocityX * deltaTime  
3     y = y + velocityY * deltaTime  
4     z = z + velocityZ * deltaTime
```

Druhá rovnice je implementována ve funkci `UpdateVelocity`, která počítá změnu rychlosti pod vlivem gravitace, větru a odporu vzduchu:

```
1 FUNKCE UpdateVelocity(wind, deltaTime):
2     // Vliv gravitace
3     gravityEffect = -GRAVITY * deltaTime
4
5     // Vliv vetru (odpor vzduchu)
6     windEffectX = (wind.Vx - velocityX) * WIND_COEFFICIENT *
7         deltaTime
8     windEffectY = (wind.Vy - velocityY) * WIND_COEFFICIENT *
9         deltaTime
10    windEffectZ = (wind.Vz - velocityZ) * WIND_COEFFICIENT *
11        deltaTime
12
13    // Aktualizace rychlosti
14    velocityX = velocityX + windEffectX
15    velocityY = velocityY + windEffectY
16    velocityZ = velocityZ + gravityEffect + windEffectZ
```

Výpočet trajektorie je ukončen, pokud:

1. střela opustí mapu – souřadnice x nebo y přesáhne hranice mapy
2. střela zasáhne terén – výška střely z je nižší než nadmořská výška terénu
3. vzdálenost mezi střelou a dronem je menší než 50 metrů – zaregistruje se zásah

3.2. Výpočet počáteční rychlosti střely

Pro výpočet trajektorie je potřeba stanovit počáteční vektor rychlosti střely podle načtených parametrů: Azimut (α), Zenit (β) a Počáteční rychlost (v_0).

Rychlost střely se rozkládá do tří složek podle rovnic:

$$v_x = v_0 \cdot \cos(\beta) \cdot \cos(\alpha) \quad (3)$$

$$v_y = -v_0 \cdot \cos(\beta) \cdot \sin(\alpha) \quad (4)$$

$$v_z = v_0 \cdot \sin(\beta) \quad (5)$$

Tyto složky určují počáteční vektor rychlosti střely $\mathbf{v}_0 = (v_x, v_y, v_z)$. Záporné znaménko u složky v_y vyplývá z orientace souřadnicového systému v simulaci, kde osa Y roste směrem dolů.

```
1 FUNKCE CalculateInitialVelocity(cannon):
2     azimuthRad = cannon.Azimuth * PI / 180
3     zenithRad = cannon.Zenith * PI / 180
4
5     horizontalSpeed = cannon.InitialVelocity * cos(zenithRad)
6
7     velocityX = horizontalSpeed * cos(azimuthRad)
8     velocityY = -horizontalSpeed * sin(azimuthRad)
9     velocityZ = cannon.InitialVelocity * sin(zenithRad)
10
11     NAVRAT (velocityX, velocityY, velocityZ)
```

Tato funkce vrací počáteční vektor rychlosti střely, který je poté použit ve funkcích UpdatePosition a UpdateVelocity.

3.3. Pohyb dronu

Tato funkce aktualizuje stav dronu v čase. Směr jeho letu je určen jednotkovým vektorem, který je vypočítán z rozdílu souřadnic dronu a cíle ve funkci SetTarget. Nejprve se ověří, zda již dron nebyl zničen. Dron postupně klesá z výšky 35 metrů nad svými počátečními souřadnicemi a rovnoměrně se snižuje až na výšku cíle (tanku). Následně se vypočítá jeho vzdálenost od cíle – pokud je dostatečně blízko, provede sebevražedný útok. V opačném případě pokračuje v pohybu směrem k cíli a provede se kontrola kolize s terénem.

```
1 FUNKCE Update(deltaTime, scenarioData, cannon):
2     POKUD IsDestroyed:
3         NAVRAT
4
5     // Vypocet vzdalenessi k cili
6     dx = targetX - X
7     dy = targetY - Y
8     dz = targetZ - Z
9     distance = sqrt(dx^2 + dy^2 + dz^2)
10
11     // Kontrola dosazeni cile
12     POKUD distance < ATTACK_DISTANCE:
13         IsDestroyed = TRUE
14         VYPIS("VYBUCH! Dron zasahl cil")
15         NAVRAT
```

```
16      // Pohyb smerem k cili
17      X = X + directionX * DRONE_SPEED * deltaTime
18      Y = Y + directionY * DRONE_SPEED * deltaTime
19      Z = Z + directionZ * DRONE_SPEED * deltaTime
20
21      // Kontrola kolize s terenem
22      CheckTerrainCollision(scenarioData)
```

3.4. Bilineární interpolace výšky

Pro přesnou detekci kolizí a plynulé vykreslování profilu byla implementována bilineární interpolace. Na rozdíl od prostého “Nearest Neighbor” přístupu (použitého v první fázi), tato metoda vypočítává výšku v libovolném bodě (x, y) na základě váženého průměru čtyř okolních bodů mřížky $(h_{00}, h_{10}, h_{01}, h_{11})$.

Vzorec pro výpočet výšky $h(x, y)$:

$$h(x, y) \approx h_{00}(1 - t_x)(1 - t_y) + h_{10}t_x(1 - t_y) + h_{01}(1 - t_x)t_y + h_{11}t_xt_y \quad (6)$$

```
1  FUNKCE GetElevation(x, y):
2      // Prevod na pozici v mrizce
3      cellX = floor(x / Dx)
4      cellY = floor(y / Dy)
5
6      // Pozice v ramci bunky (0-1)
7      tx = (x / Dx) - cellX
8      ty = (y / Dy) - cellY
9
10     // Ziskat vysky 4 okolnich bodu
11     h00 = Heights[cellY][cellX]
12     h10 = Heights[cellY][cellX + 1]
13     h01 = Heights[cellY + 1][cellX]
14     h11 = Heights[cellY + 1][cellX + 1]
15     // Interpolace v ose X
16     h0 = h00 * (1 - tx) + h10 * tx
17     h1 = h01 * (1 - tx) + h11 * tx
18     // Interpolace v ose Y
19     NAVRAT h0 * (1 - ty) + h1 * ty
```

3.5. Algoritmus predikce dopadu

Kromě ostré střelby aplikace počítá i „teoretickou“ trajektorii (červená přerušovaná čára), která hráči pomáhá s mířením. Tento výpočet provádí metoda `SimulateTrajectory`.

Na rozdíl od hlavní herní smyčky tato simulace:

- Proběhne celá v jednom okamžiku (nikoliv po snímcích).
- Používá větší časový krok ($\Delta t = 0,1$ s) pro vyšší výkon.
- **Ignoruje vektor větru** – započítává pouze gravitační zrychlení a obecný koeficient odporu vzduchu.

```
1 FUNKCE SimulateTrajectory(cannon):
2     body = seznam bodu
3     dt = 0.1 // Vetsi krok pro rychlejsi vypocet
4
5     // Rozklad pocatecni rychlosti (pouze 2D)
6     v_horiz = cannon.InitialVelocity * cos(cannon.Zenith)
7     v_vert = cannon.InitialVelocity * sin(cannon.Zenith)
8
9     distance = 0
10    height = cannon.Elevation
11
12    OPAKUJ DOKUD pocet_bodu < 2000:
13        // Aplikace odporu vzduchu a gravitace
14        horizontalVelocity += -horizontalVelocity *
15                               WIND_COEFFICIENT * dt
16        verticalVelocity += -verticalVelocity *
17                               WIND_COEFFICIENT * dt - GRAVITY * dt
18
19        distance += horizontalVelocity * dt
20        height += verticalVelocity * dt
21
22        PRIDEJ bod(distance, height) do body
23
24    POKUD height < -1: PRERUS
25
26    NAVRAT body
```

4. Vykreslení

4.1. Dělo

Funkce `cannon.Draw` vykreslí dělo na plátno. Přepočítá jeho pozici podle měřítka a natočení a následně vykreslí tělo (SVG model tanku), hlaveň (SVG model, který se zkracuje podle zenitu) a indikátor rychlosti střely.

```
1 FUNKCE DrawCannon(canvas, scale):
2     // Prepocitat pozici dela na obrazovku
3     screenX = X * scale
4     screenY = Y * scale
5
6     // Nakreslit indikator rychlosti (barva dle sily)
7     DrawVelocityIndicator(canvas, tankSize, scale)
8
9     // Rotovat platno podle azimutu
10    canvas.RotateDegrees(-Azimuth)
11
12    // Vykreslit telo tanku (SVG)
13    DrawTankBody(canvas, tankSize)
14
15    // Vypocitat vizualni zkraceni hlavne
16    shortening = cos(Zenith * PI / 180)
17
18    // Vykreslit hlaven (SVG)
19    DrawTankBarrel(canvas, tankSize)
```

4.2. Dron

Funkce `drone.Draw` vykreslí dron (SVG model nebo zástupný kruh) a zobrazí jeho informace (vzdálenost od děla, výška).

```
1 FUNKCE DrawDrone(canvas, cannon, scale):
2     POKUD IsDestroyed:
3         NAVRAT
4
5     // Prepocitat pozici dronu na obrazovku
6     screenX = X * scale
7     screenY = Y * scale
8
```

```
9      // Vykreslit dron (SVG nebo kruh)
10     DrawDrone(canvas, screenX, screenY, targetSize)
11
12     // Vykreslit text s vzdalenosti a vyskou
13     DrawInfo(canvas, cannon, screenX, screenY)
```

4.3. Mapa terénu

Pozadí herní plochy tvoří barevná mapa generovaná třídou `TerrainMap`. Výšky bodů jsou mapovány na barevnou paletu (Modrá → Zelená → Hnědá → Bílá). Barva každého pixelu je lineárně interpolována.

```
1  FUNKCE GetColorForHeight(height):
2      // Normalizace vysky do rozsahu 0..1
3      normalized = (height - MinHeight) / (MaxHeight - MinHeight)
4
5      // Urcení indexu v paletě barev
6      paletteIndex = normalized * (PaletteLength - 1)
7      index1 = floor(paletteIndex)
8      index2 = index1 + 1
9      t = paletteIndex - index1
10
11     // Lineární interpolace mezi dvěma barvami
12     color = InterpolateColor(Palette[index1], Palette[index2],
13                               t)
14     NAVRAT color
```

Pořadí	Barva	RGB	Typ terénu
1	Světle modrá	(28, 163, 236)	Voda
2	Světle zelená	(144, 238, 144)	Nížiny
3	Tmavě zelená	(34, 139, 34)	Louky
4	Olivově zelená	(107, 142, 35)	Pahorky
5	Khaki	(189, 183, 107)	Kopce
6	Béžová	(210, 180, 140)	Hory
7	Hnědá	(139, 90, 43)	Vysoké hory
8	Tmavě šedá	(105, 105, 105)	Skály
9	Šedá	(169, 169, 169)	Vrcholky
10	Bílá	(255, 250, 250)	Sníh

Tabulka 1: Barevná paleta pro vykreslení terénu

4.4. Spodní profil a grafy

Třída `TerrainProfile` vizualizuje řez terénem a trajektorie ve spodním panelu. Hlavní výzvou při vykreslování tohoto grafu je neuniformní měřítko. Zatímco osa X (vzdálenost) může zobrazovat rozsah až 10 km, osa Y (výška) zobrazuje typicky jen stovky metrů

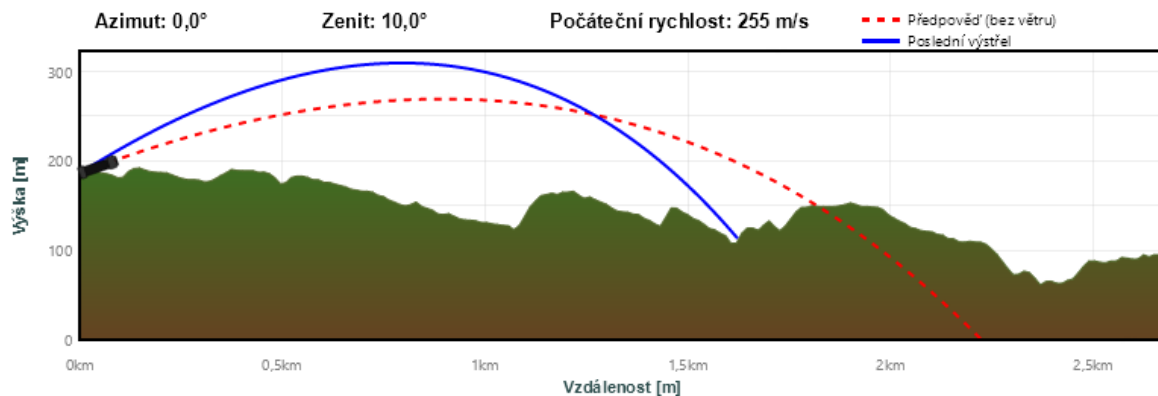
4.4.1. Korekce vizuálního úhlu hlavně

V grafu je vykreslena ikona hlavně děla. Kvůli rozdílnému měřítku os X a Y ($S_x \neq S_y$) však nelze hlaveň vykreslit pod jejím skutečným fyzikálním úhlem (např. 45° by v deformovaném grafu vypadalo jako 80°).

Metoda `DrawBarrel` vypočítává vizuální úhel θ_{vis} na základě prvních bodů simulované trajektorie po transformaci na pixely:

$$\theta_{vis} = \arctan \left(\frac{(y_2 - y_1) \cdot S_y}{(x_2 - x_1) \cdot S_x} \right) \quad (7)$$

Tím je zajištěno, že hlaveň v grafu vždy vizuálně lícuje s vykreslovanou křivkou trajektorie.



Obrázek 1: Detail spodního panelu: Vykreslení trajektorií a řez terénem

4.4.2. Terén

Pro vykreslení profilu terénu se využívá třída `SKPath`, která definuje polygon kopírující naměřené výšky. Pro vizuální hloubku je vnitřek polygonu vyplněn lineárním gradientem definovaným pomocí `SKShader`:

- **Horní barva:** Lesní zelená (RGB 34, 139, 34) – reprezentuje povrch.
- **Dolní barva:** Hnědá (RGB 101, 67, 33) – reprezentuje podloží.

Mřížka grafu a popisky os se dynamicky přizpůsobují maximální vzdálenosti dostřelu a výšce střely, aby byl graf vždy přehledný (krokování pro osu X je po 2000, 1000, 500, 200 metrech a pro osu Y 1000, 500, 200, 100).

4.5. Vítr

Funkce `wind.Draw` vykreslí indikátor větru formou kompasu se šipkou, které barva indikuje vertikální směr:

Barva	Vertikální složka	Směr
Modrá	$V_z \approx 0$	Rovně
Červená	$V_z < -2$	Dolů
Zelená	$V_z > 2$	Nahoru

Tabulka 2: Barevnost vertikální složky větru

4.6. Projektil

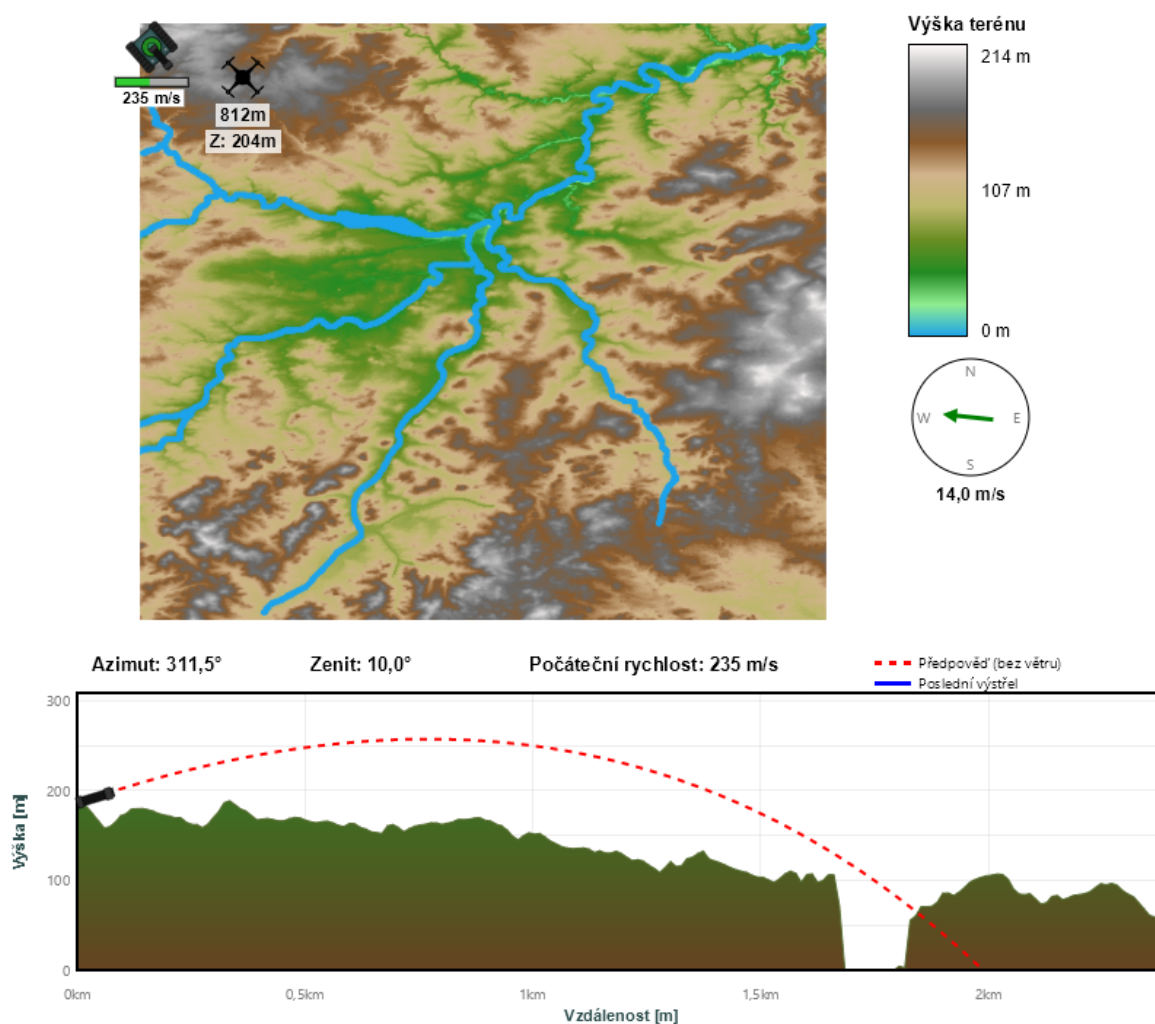
Funkce `projectile.Draw` vykreslí projektil (SVG) během letu nebo animaci exploze po dopadu.

5. Uživatelské rozhraní a ovládání

Aplikace podporuje plné interaktivní ovládání pomocí klávesnice a myši.

Akce	Klávesnice	Myš
Změna Azimutu	A / D	Tažení levým tl. v mapě (horizontálně)
Změna Zenitu	W / S	Kolečko myši / Tažení v grafu
Změna Rychlosti	+ / -	Pravé tl. myši + Kolečko
Výstřel	Mezerník	Dvojklik levým tl. na dělo

Tabulka 3: Přehled ovládání



Obrázek 2: Ukázka uživatelského rozhraní: nahoře mapa terénu legenda a kompas, dole profil řezu a parametry výstřelu.

6. Instalace a spuštění

Aplikace je vyvíjena v jazyce C# s využitím frameworku .NET.

Pro spuštění aplikace z příkazové řádky lze použít připravený skript:

```
1 Run.cmd "Cislo_scenare"
```

Aplikace využívá datové soubory jednotlivých objektů uložené ve složce `data` ve formátu `.ter`.

Parametr `"Cislo_scenare"` určuje, který scénář simulace bude načten při spuštění (0, 1, 2 nebo 3). Při spuštění bez tohoto parametru se načte výchozí scénář.

7. Implementovaná rozšíření

Nad rámec základního zadání byly implementovány následující funkce:

7.1. Proměnlivý vítr

Síla a směr větru se v čase plynule mění. Třída `Wind` generuje nové cílové vektory v náhodných intervalech (3–5 sekund) a zajišťuje plynulou interpolaci mezi stavy.

```
1  FUNKCE Update(deltaTime):  
2      // Rychlost prechodu  
3      transitionSpeed = 0.5 * deltaTime  
4  
5      // Plynulý prechod pro každou složku  
6      Vx = Change(Vx, targetVx, transitionSpeed)  
7      Vy = Change(Vy, targetVy, transitionSpeed)  
8      Vz = Change(Vz, targetVz, transitionSpeed)  
9  
10     timeToNextChange = timeToNextChange - deltaTime  
11     POKUD timeToNextChange <= 0:  
12         GenerateNewTarget()
```

7.2. Realistické modely (SVG)

Pro vizualizaci děla (tělo, hlaveň), dronu a projektilu jsou použity vektorové obrázky (SVG) namísto jednoduchých geometrických primitiv. Hlaveň děla se vizuálně zkracuje podle náklonu (zenitu), což imituje 3D pohled shora.

7.3. Kráter

Při dopadu střely do země je vytvořen kráter pomocí parabolického profilu, který v místě dopadu zůstává trvale zobrazen.

```
1 FUNKCE CreateCrater(centerX, centerY, radius, depth):
2     // Prochazeni bunek v okoli stredu
3     PRO KAZDOU bunku v okoli:
4         // Vypocet vzdalenosti od stredu
5         distance = sqrt(dx^2 + dy^2)
6
7         POKUD distance > radius:
8             POKRACUJ
9
10        // Hloubka klesa s vzdalenosti (parabolicky profil)
11        // Ve stredu = depth, na okraji = 0
12        factor = 1.0 - (distance / radius)
13        factor = factor * factor // Parabolicky profil
14        depthHere = depth * factor
15
16        // Snizit vysku terenu
17        Heights[cellY][cellX] = max(0, Heights[cellY][cellX] -
            depthHere)
```

7.4. Animace exploze

Kromě kráteru je dopad střely doprovázen dočasnou vizuální animací výbuchu, který je implementován ve třídě Explosion.

Využívá **radiální gradient** (SKShader) pro simulaci žáru. Barva se v čase dynamicky mění: od bílo-žluté (iniciace), přes oranžovou (hoření) až po tmavě šedou (rozplývající se kouř).

```
1 FUNKCE Draw(canvas):
2     progress = currentTime / DURATION
3
4     // Urceni barev podle faze (Zar -> Ohen -> Kour)
5     POKUD progress < 0.2: barvy = Bila -> Zluta
6     JINAK POKUD progress < 0.6: barvy = Zluta -> Cervena
7     JINAK: barvy = TmaveCervena -> Seda
8
9     // Vykresleni ohne s radialnim gradientem
10    Shader = CreateRadialGradient(Center, Colors=barvy)
11    DrawCircle(Filled, Shader)
```

8. Použité zdroje a nástroje

Při realizaci semestrální práce byly využity následující knihovny a zdroje:

8.1. Knihovny a Frameworky

- **Avalonia** – .NET framework použitý pro vytvoření uživatelského rozhraní.
- **SkiaSharp** – 2D grafická knihovna využitá pro vykreslování herní plochy a profilu terénu.
- **Svg.Skia** – Knihovna pro načítání a vykreslování vektorové grafiky (SVG) v prostředí SkiaSharp.

8.2. SVG modely

- **Model dronu:** Staženo z databáze *SVG Silh* (licence Public Domain).
Dostupné z: <https://svgsilh.com/image/2025680.html>
- **Ostatní modely (Tank, Dělo, Projektil):** Byly vygenerovány s pomocí nástroje Google Gemini.